

PROMPT IA 11

Admin Backoffice complet - NOMA

Next.js + TypeScript + Tailwind CSS + shadcn/ui + Radix UI

Source visuelle obligatoire : utiliser le Design Pack Admin NOMA fourni comme reference visuelle et ergonomique. Le developpement doit conserver la meme identite NOMA que le Storefront tout en adoptant une interface dense, desktop-first et operationnelle.

Backend authoritative - RBAC - audit - performance - accessibilite - GitOps

1. Role de l'IA

Tu es Lead Frontend Engineer / Product Engineer. Tu dois implementer l'Admin Backoffice complet dans le monorepo existant, en respectant strictement les contrats backend, la securite, le RBAC et le Design Pack Admin NOMA fourni.

Ne redessine pas l'Admin. Le Design Pack est la source visuelle de reference. Corrige seulement les incoherences fonctionnelles/accessibilite evidentes sans changer l'identite visuelle.

2. Sources de verite

Le developpement doit utiliser les sources suivantes par ordre de priorite :

- 1. Contrats backend et regles metier validees
- 2. Design Pack Admin NOMA
- 3. Shared NOMA tokens du Storefront
- 4. Design system Tailwind/shadcn/Radix
- 5. Conventions Next.js du monorepo

En cas de contradiction, la securite, la logique metier et les contrats backend priment sur une maquette raster.

3. Stack imposee

Next.js App Router
TypeScript strict
Tailwind CSS
shadcn/ui
Radix UI
React Server Components par default
Client Components uniquement quand necessaires
CSS variables / design tokens
fetch / Server Actions lorsque pertinent
OpenTelemetry frontend/BFF
Playwright pour E2E

Interdits comme stack principale : Bootstrap, Materialize, MUI, runtime CSS-in-JS lourd.

4. Identite visuelle NOMA

Reutiliser les tokens du Design Pack :

Token	Valeur
Ivory	#F7F3EC
White	#FFFFFF
Charcoal	#17202A
Electric Blue	#2764FF
Coral	#FF6B55
Muted Gray	#8A9099
Radius reference	12 px
Spacing base	8 px

Ajouter les couleurs fonctionnelles Admin success/warning/error/info sans modifier les couleurs de marque partagees.

5. Layout global

Topbar
- global search / command palette
- alerts

- current environment badge
- current user / role

Sidebar

- Dashboard
- Products
- Catalog
- Pricing
- Inventory
- Orders
- Payments
- Returns
- Shipping
- Reviews
- Fraud
- Users
- Audit
- Settings

Main workspace

- breadcrumb
- title/actions
- filters/tabs
- content

Sidebar compacte/expandable. L'environnement PRODUCTION/PREVIEW/LOCAL doit toujours etre visible.

6. Authentification et autorisation

Utiliser Keycloak/OIDC. Le frontend peut cacher/desactiver une action selon les claims, mais le backend reste toujours authoritative.

```
super_admin
catalog_manager
inventory_manager
order_manager
finance_manager
support_agent
customer
```

Les permissions fines telles que product:write, inventory:adjust, order:refund doivent etre verifiees par le backend/OPA. Une UI cachee n'est jamais un controle de securite.

7. Dashboard

Implementer l'ecran Dashboard conforme a la maquette : KPI utiles, alertes actionnables, activity feed, commandes recentes et graphiques temporels.

Eviter les requetes multiples non coordonnees. Agreger via BFF/read models et charger les panels lourds en streaming/Suspense si necessaire.

8. Products

8.1 Products List

Table dense : image, product, SKU, category, price, stock, visibility, status, updated, actions. Ajouter recherche, facettes, tri, colonnes configurables, bulk actions, pagination curseur et vues sauvegardees.

8.2 Product Editor

Sections : General, Media, Variants/SKU, Attributes, Pricing, Catalog placement, Inventory summary, Search preview, Audit history. Gerer dirty state, validation, save/publish, permission denied et conflits de version.

9. Catalog et Pricing

Catalog gere categories/collections/navigation/merchandising. Pricing gere les prix commerciaux. L'Admin ne doit pas fusionner les ownerships backend.

Les formulaires envoient des commandes aux APIs metier correspondantes et affichent les erreurs backend de facon actionnable.

10. Inventory

Vue stock : SKU, produit, available, reserved, on hand, reorder level, site/warehouse, status. Ajustement via drawer/dialog avec reason obligatoire, impact explicite et audit actor/action/result.

Aucun ajustement de stock direct en base. Toute modification passe par Inventory Service.

11. Orders

11.1 Orders List

Filtres rapides : Pending, Confirmed, Paid, Processing, Shipped, Delivered, Cancelled, Refund requested, Fraud review.

11.2 Order Detail

Reproduire la maquette NOMA : summary, price snapshot, tax snapshot, payment, risk, stock reservation, shipment, tracking timeline, invoice, returns, refunds, notifications et audit.

Les actions sensibles (Cancel, Refund, Create Return, Resend Invoice) doivent etre permission-aware, clairement distinguees et confirmees selon le niveau de risque.

12. Payments et Refunds

Afficher amount, currency, PSP, PSP reference, authorization, capture, refunds, risk, timeline et idempotency reference. Ne jamais afficher/stocker PAN complet ou CVV.

Refund dialog : amount, reason, impact summary, confirmation. Backend valide montant, statut et droit d'action avant toute confirmation frontend.

13. Returns

REQUESTED -> APPROVED/REJECTED -> LABEL_CREATED -> IN_TRANSIT
-> RECEIVED -> INSPECTED -> REFUND_PENDING -> REFUNDED -> CLOSED

Afficher timeline, items, reason, shipping return status, inspection, restock decision, refund et credit note.

14. Shipping et Tracking

Afficher carrier, service, shipment status, tracking number, events, ETA, exceptions et actions permises. Les appels carrier passent uniquement par Shipping/Tracking backend, jamais directement depuis le navigateur.

15. Fraud/Risk

Console REVIEW avec score, regles declenchees, signaux disponibles, historique et decisions ALLOW/REVIEW/DENY. Ne jamais presenter le score comme une verite absolue. Toute decision humaine est auditee.

16. Reviews Moderation

Liste et detail des avis avec rating, product, customer reference, reports, status. Actions Publish/Hide/Reject/Flag avec raison si necessaire.

17. Users et permissions

Gestion des comptes/admin roles via APIs/IAM autorisees. Les changements de role sont sensibles, audites et peuvent demander step-up MFA selon la policy.

18. Audit

Vue read-only structuree : actor, action, resource, timestamp, result, correlation_id, trace_id, change_id.
Recherche/filtres/export selon permissions. Aucun bouton de modification de l'audit.

19. Etats UI obligatoires

```
default
hover
focus-visible
active
disabled
loading / skeleton
empty
error
success
partial data
permission denied
stale data
conflict / version changed
```

Ne jamais utiliser uniquement la couleur pour communiquer un statut.

20. Performance

Admin desktop-first mais performant. RSC/SSR par default, JS uniquement lorsque l'interactivite le necessite. Dynamic imports pour tables/graphes lourds, virtualisation lorsque la volumetrie le justifie, pagination curseur, debounce/AbortController pour recherche.

Mesurer bundle, LCP, INP, CLS. Ne pas ajouter de librairie lourde sans benefice mesurable.

21. Data fetching et cache

Le navigateur ne doit pas appeler directement chaque microservice. Utiliser Next.js comme BFF lorsque cela reduit les round-trips et facilite l'autorisation/composition.

```
Browser -> Next.js BFF/RSC -> APIs/read models
-> backend authoritative
```

Les donnees d'ecriture ne sont jamais considerees confirmees avant reponse backend authoritative.

22. Temps reel

Utiliser polling raisonnable ou SSE pour alertes/statuts qui le justifient. WebSocket uniquement si bidirectionnel reellement necessaire. Eviter les boucles de polling agressives.

23. Accessibilite

Cible WCAG 2.2 AA : navigation clavier, focus visible, semantic tables, labels, annonces erreurs, contrastes, touch targets suffisants, reduced motion et lecteurs d'ecran.

24. Responsive

Admin desktop-first. Implementer desktop large, desktop standard, tablette et mobile d'urgence. Mobile priorise consultation, alertes, recherche et quelques actions simples. Les workflows complexes peuvent explicitement recommander desktop.

25. Tests

Type	Minimum
Unit/Component	composants, hooks, formatters, permission helpers

Type	Minimum
Contract	schemas API, erreurs, pagination, actions
E2E Playwright	login, product edit, inventory adjust, order detail, refund, return, fraud decision, audit
Accessibility	axe ou equivalent + keyboard smoke tests
Visual regression	ecrans critiques compares au Design Pack
Performance	bundle budgets + Core Web Vitals lab/RUM selon environnement

26. Securite frontend

CSP stricte, cookies HttpOnly/Secure/SameSite quand applicables, aucune cle/secret dans le bundle, prevention XSS, protection CSRF selon pattern choisi, validation backend systematique et logs sans PII inutile.

27. Observabilite

Instrumenter navigation, erreurs client, appels BFF et actions admin sensibles avec correlation_id/trace_id. Ne jamais enregistrer secrets, tokens complets ou donnees de paiement sensibles.

28. Structure recommandee

```
apps/admin/
app/
components/
ui/
domain/
features/
products/
catalog/
pricing/
inventory/
orders/
payments/
returns/
shipping/
reviews/
fraud/
users/
audit/
lib/
api/
auth/
permissions/
telemetry/
styles/
tests/
```

Factoriser les composants partageables sans creer un framework interne excessif.

29. Ordre d implementation

1. Design tokens + app shell + auth
2. Dashboard
3. Products List
4. Product Editor
5. Orders List
6. Order Detail
7. Inventory
8. Payment/Refund
9. Returns/Shipping/Tracking
10. Fraud/Reviews

- 11. Users/Audit
- 12. Responsive + accessibility
- 13. E2E + visual regression + performance
- 14. CI/CD + deployment

30. Definition of Done

- Le rendu respecte le Design Pack Admin NOMA.
- Tous les ecrans critiques utilisent les memes tokens/composants.
- Les actions sensibles sont protegees par permissions et confirmations appropriees.
- Le backend reste authoritative pour toute decision et tout etat final.
- Aucune DB ni API provider externe n'est appelee directement depuis le navigateur.
- Les workflows Product, Inventory, Order, Refund, Return et Fraud ont des E2E passants.
- Les ecrans critiques passent accessibilite et visual regression.
- Le bundle et les Core Web Vitals respectent les budgets fixes.
- Le build TypeScript strict, lint et tests passent en CI.
- Le deploiement est produit via le workflow GitOps existant.

Ne termine pas le travail sur des maquettes statiques. Le livrable est termine quand les workflows fonctionnent contre les APIs backend, que les permissions sont revalidees cote backend et que les tests E2E critiques passent.